

原 理 · 自 学 · 8 天

从零理解 大语言模型

A Bottom-Up Course on How LLMs Really Work

基于 Sebastian Raschka 《从零构建大语言模型（LLM）》设计

适合对象：刚高考完、几乎零编程基础的 18 岁学生

目标：吃透 LLM 的底层原理，并能在此基础上看懂当下纷繁芜杂的 AI 发展，形成根本性的判断力

贯穿全程的一条主线

所有 AI，归根结底都在预测下一个词。

写在最前面：这门课和你想的不一样

这本书原本是给会写 Python 的人看的，主线是"亲手敲代码造一个 GPT"。但对一个零基础、自学为主、最看重"看懂原理"的人来说，**照着书一行行默写代码既不现实，也不是重点。**

所以这门课重新定了三条规矩，请一开始就记住：

1. **代码"读懂 + 跑通"即可，写不出来很正常。** 你不需要能凭空写出任何一段代码。你需要的是：把官方的在线代码（Colab notebook）一格一格跑通、读懂每一块在干什么、为什么这么干，偶尔改几个数字看看结果怎么变。仅此而已。
2. **每天只抓一个"大想法"。** 8 天里有 8 个核心概念。其余细节都是为这个大想法服务的。如果某天你只记住了那一句话直觉，这天就没白过。
3. **始终抓住一条主线。** 整本书、整个 AI 领域，都在围绕一件事：**预测下一个词**。你现在用 ChatGPT，它回答你的本质就是——不停地预测下一个词。把这条线攥紧，后面所有花里胡哨的东西都会归位。

给组织者/家长的话：这份讲义可以当老师讲、也可以让孩子自己读。每天的结构是固定的——**一句话直觉 → 读什么 → 动手（可选）→ 原理讲透 → 连接现实 → 自检**。"连接现实"那一栏是这门课真正的价值所在，别跳过。松散自学的话，建议每天投入 2-4 小时，8 天走完；想压成 7 天，把第 7 天并进第 6、8 天即可。

开课前：把工具准备好（30 分钟，一次性）

零基础不用装任何软件，全程在浏览器里完成：

- **Google Colab** (colab.research.google.com)：免费的在线编程环境，自带免费 GPU，打开网页就能跑代码，不用配置任何东西。我们造的模型很小，免费版完全够。
- **本书配套代码仓库：**GitHub 上搜 `rasbt/LLMs-from-scratch`。作者把每一章的代码都做成了可以直接在 Colab 打开的 notebook。这是你"动手"的全部素材。
- **本书 PDF：**作为主线和查阅。

你唯一需要学会的操作：在 Colab 里**点一下某个代码格子左边的播放按钮，让它运行**。看到下面冒出结果，就成功了。会这个，就够开始了。

第 1 天 大地图：一句话看懂整个 LLM

一句话直觉：大语言模型就是一个**超级下一个词预测器**——给它一段文字，它猜下一个最可能的词；把这个动作重复几百次，就成了你看到的"回答"。

对应书中

第 1 章（了解大语言模型）+ 附录 A（PyTorch 入门，只看 A.1-A.2 的概念，不必啃细节）

今天要建立的三个认知

1) 整个领域只有一件事：**预测下一个词**。"今天天气真不____"，你脑子里立刻冒出"错/好"。LLM 干的就是这个，只不过它见过的文字多到难以想象，所以能接得很像人。它回答你一个长段落，是因为它**预测了下一个词，把这个词加回去，再预测下一个，循环往复**。没有别的魔法。把这句话刻进脑子里，它会贯穿这门课的每一天。

2) 造一个 LLM 分两大阶段。

- **预训练 (Pre-training)**：拿海量文本（约等于半个互联网）让模型反复做"猜下一个词"的练习。练完，它学会了语言和大量世界知识，但还很"野"——只会续写，不会听话。这一步极贵（要烧很多算力）。
- **微调 (Fine-tuning)**：在预训练好的模型上，用一小批精心准备的数据再轻轻调教，让它学会特定行为（比如"听懂指令、好好回答"）。这一步便宜，但决定了模型**"懂不懂事"**。ChatGPT 之所以是 ChatGPT，关键就在这一步。

记住这张图：**海量文本** → (预训练, 烧钱) → 会语言但很野的**"基础模型"** → (微调, 调教) → 听话的助手。
后面 6 天就是在拆解这两个箭头里到底发生了什么。

3) 今天的所有大模型，都是同一个家族。GPT、Claude、Gemini、DeepSeek、Qwen、Llama..... 名字五花八门，但底层都用同一种架构叫 **Transformer**（2017 年提出），都遵循上面那两个阶段。**这意味着：你把这本书里那个小模型搞懂了，你就懂了它们全部的骨架**。它们的差别主要是"更大、喂更多数据、调教方式不同"，而不是"原理不同"。这是你能"看懂整个 AI 世界"的根本原因。

极简数学直觉（用你高考已有的底子，10 分钟）

后面会反复用到三个概念，现在先建立直觉，不用会算：

- **向量 = 一串数字 = 空间里的一个点**。高考里你学过平面向量（2 个数）、空间向量（3 个数）。LLM 里一个词会被表示成几百个数（比如 768 个）——你想象不出 768 维空间没关系，**只要记住"一个词 = 高维空间里的一个点，意思相近的词，点也靠得近"**。

- **矩阵乘法 = 加权求和**。神经网络的每一层，本质就是"把输入的一串数，各自乘上一些权重再加起来，得到新的一串数"。那些"权重"就是模型要学的东西（叫**参数**）。你不用会手算矩阵乘，只要知道：**模型学习 = 不停地调整这些权重数字**。
- **梯度 = 往哪边挪能更对**。你高考学过用**导数求函数极小值**（导数为 0 的地方可能是谷底）。模型训练就是这件事的放大版：把"模型错得多严重"写成一个函数，然后顺着导数（在多维情况下叫**梯度**）一点点往"错得更少"的方向挪权重，挪几十亿次。**训练 = 滚下山找谷底**。

动手（可选，30 分钟）

打开 Colab，跑通第 1 章对应的 notebook（或附录 A 的张量小例子）。目标只有一个：**感受"运行一段代码、看到结果"是什么体验**，顺便看看"一串数字（张量）"长什么样。看不懂细节完全没关系。

连接现实

为什么 2026 年的 AI 新闻里全是"GPT-5.5、Claude、Gemini、DeepSeek、Qwen 在几天内扎堆发布"？因为它们共享同一套配方（Transformer + 预训练 + 微调），配方公开，所以**谁有更多算力、更好的数据和调教，谁就能往前冲一步**。开源模型（DeepSeek、Qwen、Llama）能追平闭源巨头，也正是因为这套底层配方没有秘密——而这套配方，正是你接下来 7 天要亲手拆开的。

自检（能用自己的话回答就过关）

1. 用一句话说清楚：ChatGPT 回答你一段话，机制上到底在做什么？
2. 预训练和微调分别解决什么问题？哪个贵、哪个决定"懂不懂事"？
3. 为什么说"搞懂这本书里的小模型，就懂了所有大模型的骨架"？

第 2 天 文字怎么变成数字

一句话直觉：计算机只会算数字，不认识字。所以第一步永远是把文字切成块、再把每一块变成“高维空间里的一个点”。**意义，就是位置。**

对应书中

第 2 章（处理文本数据），全章

原理讲透

模型不能直接吃文字，要经过一条流水线：

- 1) **分词 (Tokenization)：把文字切成"token"。**一段话被切成一个个小块 (token)。token 不完全等于“词”——有时是一个完整的词，有时是半个词、一个字、甚至一个标点。比如英文 "unbelievable" 可能被切成 "un + believ + able"。这种切法叫 **BPE (字节对编码)**，好处是：用有限的一套小块，能拼出几乎任何词，包括没见过的新词。
- 2) **转成 ID：每个 token 查表得到一个编号。**模型内部有一本“字典”，每个 token 对应一个整数编号 (token ID)。文字到这里就变成了一串数字。
- 3) **词嵌入 (Embedding)：把编号变成“高维空间里的点”。**光有编号还不行（编号 5 和编号 6 并不意味着意思相近）。所以再把每个编号映射成一串浮点数（比如 768 个数）的向量——这就是**嵌入**。神奇之处在于：**训练之后，意思相近的词，它们的向量会自动靠得很近**（“国王”和“女王”、“猫”和“狗”在这个空间里是邻居）。**意义被编码成了空间中的位置。**
- 4) **位置编码 (Positional Encoding)：告诉模型词的先后顺序。**“狗咬人”和“人咬狗”用的词一样，意思完全相反。所以还要额外给每个 token 加上“它排在第几位”的信息。

走完这条流水线，一段文字就变成了一摞带位置信息的向量，可以喂给模型了。

动手（可选，40 分钟）

跑第 2 章 notebook。重点感受两件事：① 同一句话被切成了哪些 token（试试中英文混合、试试生僻词）；② 一个词的嵌入向量长什么样（就是一长串小数）。改几个输入，看 token 怎么变。

连接现实（这一栏开始变得重要）

有了今天的原理，很多“AI 怪现象”瞬间不再神秘：

- **“上下文窗口 128K”是什么意思？**指模型一次最多能读多少个 token。token 不是字数——一段中文的 token 数和字数不一样。

- **为什么 API 按 token 收费、为什么长对话会"忘事"?** 因为模型处理的单位是 token，而它一次能装下的 token 有上限，超了就得丢掉最早的内容。
- **为什么模型有时连"strawberry 里有几个 r"都数不清?** 因为它看到的不是一个个字母，而是被切好的 token 块，字母层面的信息在分词时就被"打包"了。理解了分词，你就不会再觉得这是"AI 变笨了"，而知道这是机制使然。

自检

1. token、token ID、嵌入向量，三者分别是什么？流水线顺序是怎样的？
2. 为什么需要嵌入，光有 token ID 不够吗？"意义即位置"怎么理解？
3. 用今天学的，解释为什么模型会数错单词里的字母。

第 3 天 注意力机制：整本书的心脏

一句话直觉：句子中的每个词都“环顾全场”，从其他词那里把和自己相关的信息**拉过来**，从而理解自己在这个具体语境里到底是什么意思。这个“环顾并拉取”的动作，就叫**注意力**。

对应书中

第 3 章（编码注意力机制），全章——这是全书最关键、也最值得多花时间的一章

为什么这一天最重要

注意力（Attention）是 Transformer 真正的核心创新，是 2017 年那篇改变一切的论文《Attention Is All You Need》的主角。**搞懂了注意力，你就摸到了现代 AI 的命门。** 哪怕代码一行看不懂，只要把下面的直觉建立起来，这一天就值了。

原理讲透

问题：词的意思依赖语境。“我去银行取钱”和“我坐在河的银行边”——同一个“银行”，意思天差地别。模型怎么知道该取哪个意思？答案是：让“银行”去看它周围的词（“取钱” vs “河边”），据此调整自己的含义。这就是注意力要解决的事。

机制：用三个角色来理解（这是全书最值得记的一组比喻）——

- **Query（查询）** = “我在找什么”：每个词发出一个“我想了解的信息”信号。
- **Key（键）** = “我能提供什么”：每个词同时挂出一个“我这儿有什么信息”的标签。
- **Value（值）** = “选中我，我就给你这些”：每个词还准备了一份“真正要传递的内容”。

流程：每个词拿自己的 Query 去和所有词的 Key 比对，谁匹配得好，就多分一点“注意力”给谁；然后按这个注意力比例，把大家的 Value 加权混合起来，作为这个词更新后的理解。“银行”的 Query 和“取钱”的 Key 高度匹配，于是它大量吸收了“取钱”的 Value，含义就被确定为“金融机构”。

两个关键细节：

- **因果掩码（Causal Mask）**：因为模型的任务是“预测下一个词”，所以在处理第 5 个词时，**绝不能让它偷看第 6、7 个词**（那是答案）。掩码就是把“未来”挡住，确保它只能根据已经出现的词来预测。
- **多头注意力（Multi-Head）**：与其让模型只用一种方式“环顾全场”，不如让它同时用很多套独立的注意力（比如 12 套），各看各的关系——有的关注语法、有的关注指代、有的关注主题。最后把它们的结果合起来。**这就是“多头”**。

动手（可选，可深可浅）

跑第 3 章 notebook。如果只想浅尝：看一个简单例子里，“注意力权重”是怎么从一句话里算出来的，哪个词给了哪个词多少注意力（通常能可视化成一张热力图）。光看那张图，就能直观感受“词在互相看”。

连接现实

- **为什么 Transformer 颠覆了之前的所有方法？** 因为注意力让每个词能一步直达句子里任何其他词，不管隔多远——这比老方法（一个词一个词顺着传）强太多，也更适合大规模并行计算（能喂给 GPU 一起算），于是模型才得以越做越大。
- **为什么“长上下文”很贵、是技术难点？** 因为每个词都要和其他每个词两两比对， n 个词就要算大约 $n \times n$ 次。词数翻倍，计算量翻四倍。这就是为什么“128K、1M 上下文窗口”是各大公司炫耀的工程成就——它们想办法绕开了这个 n^2 的成本。
- **当你听到“这个模型注意力机制做了改进”**——你现在知道那指的是动了今天这套 Q/K/V 的算法，目的通常就是为了**更长、更快、更省**。

自检

1. 注意力到底在解决什么问题？用“银行”的例子说一遍。
2. Query / Key / Value 各自的角色是什么？（这是必须能背出的一组比喻）
3. 因果掩码为什么必须存在？多头注意力为什么有用？
4. 为什么长上下文又贵又难？这和注意力的算法有什么关系？

第 4 天 把 GPT 拼起来

一句话直觉：一个 GPT，就是把"注意力 + 一个小的前馈网络"打包成一块**积木**，然后把同样的**积木重复堆叠很多层**。所谓"参数"，就是这些积木里那一大堆可调的数字。

对应书中

第 4 章（从头实现一个 GPT 模型来生成文本），全章

原理讲透

昨天的注意力是核心零件，今天把它装进整机。一个 Transformer 积木块（block）里有这么几样东西：

- **注意力层**（昨天讲的，负责"词之间互相看"）。
- **前馈网络（Feed-Forward）**：一个小型神经网络，对每个词单独做一次"加工提炼"。如果说注意力是"词之间交流"，前馈就是"每个词自己消化"。中间用一个叫 **GELU** 的非线性函数，让模型能表达复杂的关系（没有非线性，再多层也等于一层）。
- **残差连接（Residual / 快捷连接）**：给信息修一条"高速公路"，让它可以跳过某层直接往后传。这样堆很多层也不会"信息丢失/训练崩掉"。这是能把模型堆到几十上百层的关键工程技巧。
- **层归一化（LayerNorm）**：每过一层就把数值"重新标准化"一下，保持稳定，免得越乘越爆炸或越缩越没。

把上面打包成一块**积木**，然后堆 **N 块**（GPT-2 small 堆 12 块，大模型堆几十上百块）。最后接一个输出层，把最终的向量翻译回"下一个词是什么"的概率。这就是一个完整的 GPT。今天你会让它**生成文本**——虽然没训练过，吐出来的是乱码，但**架构是真的、完整的**。

动手（可选，40 分钟）

跑第 4 章 notebook，让未训练的 GPT 生成一段（乱码）文本。重点感受：① 同一块积木被重复堆叠；② 打印一下模型的"参数量"（多少个可调数字）。改一下"堆几层"，看参数量怎么变。

连接现实

- **"参数量"到底是什么？** 就是模型里所有这些可调数字的总数。GPT-2 有 1 亿多到 15 亿，今天的大模型动辄几百亿、几千亿。**参数越多，模型能记住和表达的东西越多**（但也越贵）。
- **GPT-2 / GPT-3 / GPT-4 / GPT-5 的本质差别是什么？** 不是原理不同，而是**同一块积木堆得更多、喂的数据更多、训练得更久**。这就是著名的"规模法则（Scaling Law）"的雏形——明天细讲。理解了这点，你看任何"参数量创新高""又一个更大的模型"的新闻，都会明白那是在沿着同一条已知的路往前推。

- **"MoE（专家混合）"是什么？**（先埋个钩子，第 7、8 天展开）就是把今天这个"前馈网络"换成很多个并排的小网络（专家），每个词只走其中几个。这样能在不成比例增加计算的前提下，把参数量做得极大。你现在已经知道它动的是积木里的哪个零件了。

自检

1. 一块 Transformer 积木里有哪几个部件？各自大致干什么？
2. 残差连接和层归一化为什么重要（为什么没它们就堆不深）？
3. "参数"到底指什么？为什么 GPT 越新参数越多，但原理没变？

第 5 天 预训练：模型怎么"学会"整个世界

一句话直觉：把半个互联网的文本，通过亿万次"猜下一个词"的练习，**压缩进模型的几十亿个参数里**。文本天然自带答案（下一个词就是标准答案），所以不需要人工标注——这叫**自监督**。

对应书中

第 5 章（在未打标签的数据上进行预训练），全章

原理讲透 —— 这是"模型为什么这么神"的答案

1) 训练循环长什么样（昨天的"滚下山"在这里落地）：

- **前向：**给模型一段文本，让它预测每个位置的下一个词。
- **算损失 (Loss)：**用一个叫**交叉熵**的指标，衡量"模型的预测"和"真实的下一个词"差多少。差得越多，损失越大。
- **反向 + 更新：**算出每个参数"该往哪边挪能让损失变小"（梯度），然后挪一小步。
- **重复**几百万、几十亿次。模型的参数就这样被一点点雕刻成形。

2) **为什么这一步能让模型"懂世界"？** 为了把"下一个词"猜准，模型被迫学会语法、事实、推理、常识——因为要接好"水的沸点是 100____"，它就得"知道"摄氏度和水的性质。**没人专门教它这些知识，它是为了完成"猜词"任务而被逼出来的。** 这就是为什么一个看似简单的目标，能催生出惊人的能力。

3) **"自监督"为什么是关键突破？** 传统机器学习要人工标注大量数据（成本极高）。但语言模型不用——任何一段文本，把前面遮住一部分，剩下的就是题目，被遮的就是答案。互联网上海量的免费文本，瞬间都成了练习题。这是 LLM 能做到这么大的根本原因之一。

4) **解码策略（模型怎么"说话"）：** 预测出的是"每个词的概率"。怎么从概率里选词？

- **温度 (Temperature)：**调高 → 更随机、更有创意但可能跑偏；调低 → 更保守、更确定。
- **Top-k / Top-p：**只在概率最高的几个候选里挑，避免选到离谱的词。这就是你在很多 AI 工具里看到的"temperature"滑块背后的东西。

5) **站在巨人肩上：**本章最后会教你直接加载 OpenAI 公开的 GPT-2 权重——也就是说，别人花大钱预训练好的"大脑"，你可以直接下载来用。这正是现实中绝大多数人和公司的做法：**不自己预训练，而是拿现成的模型来微调。**

动手（可选）

跑第 5 章 notebook，看损失曲线怎么随训练下降（直观感受"滚下山"）；调一下温度，看同一个模型生成的文本怎么从"刻板"变"放飞"。

连接现实

- **规模法则 (Scaling Law)**：实践发现，**参数更多 + 数据更多 + 算力更多**，损失会可预测地下降，能力会随之上一个台阶——甚至"涌现"出没专门训练过的新能力（比如做数学题、写代码）。这就是过去几年"暴力堆规模"路线的理论依据，也是为什么算力（GPU/芯片）成了国家级的战略资源。
- **"基础模型 (base model)"为什么很怪？** 因为预训练完的模型只会"续写"，不会"对话"。你问它问题，它可能不回答，反而续写出更多问题。它必须经过明天讲的微调，才变得"懂事"。
- **为什么 2026 年大家说"预训练的红利在见顶，重心转向推理时计算"？** 因为一味堆规模的边际收益在递减，且受限于数据和能源。于是前沿转向了别的方向（推理模型、智能体）——这些第 8 天会接上。你现在已经站在能理解这个转变的位置上了。

自检

1. 完整描述一遍训练循环（前向 → 损失 → 梯度 → 更新）。
2. "自监督"为什么是关键突破？为什么"猜下一个词"能逼出世界知识？
3. 温度调高调低分别意味着什么？
4. 用自己的话讲讲规模法则，以及为什么"算力是战略资源"。

第 6 天 微调：从"会接话"到"听话"

一句话直觉：拿预训练好的模型（已经会语言了），用一小批精心准备的数据再轻轻调教，就能改变它的行为。其中**指令微调**这一步，把一个"自动续写机"变成了"听话的问答助手"——**这就是 ChatGPT 诞生的那一步。**

对应书中

第 6 章（微调分类器）+ 第 7 章（指令微调）——两章可以合在一天，因为机器零件（数据、训练循环）你前几天已经见过了，今天看的是**怎么用同一套机器达成不同目的**。

原理讲透

微调的本质：不重新造模型，而是在"已经很懂语言的大脑"上，用少量数据继续训练几步，把它的行为往某个方向轻轻掰。因为底子已经很好，**只需很少的数据和算力，就能见效。**

两种微调，看清"调教"的两个层次：

1) **分类微调（第 6 章）——让模型"做判断题"。**例子：判断一封邮件是不是垃圾邮件。做法：把模型最后的"预测下一个词"的输出头，换成"输出'是/否'两个类别"的头，然后用一批标好的邮件训练几步。**模型理解语言的能力直接复用，只是产出形式从"续写"变成了"分类"。**这让你看清：同一个预训练大脑，能被改造去干各种具体活儿。

2) **指令微调（第 7 章）——让模型"听懂人话、好好回答"。**← 今天的重点 例子：给模型成千上万条"指令—理想回答"的样例（"把这句话翻译成英文 → ..." "总结这段话 → ..."），训练它模仿这种"接到指令就给出合适回答"的模式。

这一步极其关键，请停下来体会：GPT-3（2020）已经很强，但它只是个"续写机"，不好用。直到 OpenAI 用指令微调（再加上明天讲的 RLHF）把它调教成会对话、肯帮忙的样子，才有了 2022 年底引爆全球的 ChatGPT。**底层模型没变多少，变的是这一层调教。**你现在亲手摸到了"AI 从实验室玩具变成全民工具"的那个转折点。

动手（可选）

跑第 6、7 章 notebook，亲眼看一个模型在指令微调前后，对同一个问题的回答有多大差别。这个对比往往最有冲击力。

连接现实

- "对齐 (Alignment)" 在机制上是什么？听起来玄乎，但你现在知道了：它本质就是预训练之后，用各种方式（指令微调、人类偏好反馈……）继续调教模型的行为，让它有用、诚实、无害。不是什么神秘的东西，就是"在大脑上做行为塑造"。
- 为什么同一个开源底座（比如 Llama、Qwen），社区能调出几百个不同性格、不同专长的版本？因为预训练成本高、只有少数巨头做得起，但微调便宜，人人都能在现成底座上调出自己想要的模型。理解了这点，你就看懂了整个"开源模型生态"为什么这么繁荣。
- SFT vs RLHF（明天展开）：今天学的指令微调叫 SFT（监督微调），是"给它看标准答案让它模仿"。明天会讲 RLHF——"让它生成几个答案，由人/AI 判断哪个更好，再据此调整"。两者配合，才有今天聊天助手的体验。

自检

1. 微调为什么便宜、为什么少量数据就有效？
2. 分类微调和指令微调分别在干什么？产出形式有何不同？
3. 为什么说指令微调是"ChatGPT 诞生的那一步"？底层模型变了吗？
4. 用你自己的话解释"对齐 (alignment)" 到底是什么。

第 7 天 工程现实，和书没讲的那些拼图

一句话直觉：到这里，你已经走完了“造一个能用的 LLM”的**主干**。真实世界里的那些热词，几乎都是在这条主干上**加的枝**——而你现在有能力看清每根枝长在哪。

对应书中

附录 E (LoRA 参数高效微调) + 附录 D (训练技巧) + 第 7.9 节 (“下一步是什么”)

原理讲透

1) LoRA / PEFT：为什么微调可以“又快又省”。完整微调要动模型全部几十亿个参数，很贵。LoRA 的聪明之处：冻结原模型不动，只在旁边加一小撮新参数来训练（好比不重装整台发动机，只加个小外挂芯片）。结果：用极少的资源就能微调超大模型，普通人用一张消费级显卡就能定制大模型。这就是为什么“人人都能微调”成了现实。

2) 这本书故意没讲的部分（要心里有数）：这本书教的是 LLM 的**核心骨架**，但为了聚焦，它略过了真实产品里的好几块，包括：

- **RLHF（人类反馈强化学习）**：在指令微调之后，进一步用“人类更喜欢哪个回答”的偏好信号来打磨模型。
- **推理模型 / 思维链**：让模型在回答前先“想一长串草稿”。
- **RAG（检索增强）**：不把所有知识塞进参数，而是临时查资料喂给模型。
- **智能体（Agent）**：让模型能调用工具、看结果、再决定下一步，形成循环。
- **多模态**：让模型不只处理文字，还能看图、听声。

这些**不是另起炉灶的新原理**，全都建立在你这 6 天学的基础上。明天（第 8 天）会把它们一个个接到你已有的知识上——那是这门课的高潮。

动手（可选）

跑附录 E 的 LoRA notebook，对比“完整微调”和“LoRA 微调”在参数量、速度上的差距。直观感受“为什么人人都能微调”。

连接现实

- **为什么 DeepSeek、Qwen、Llama 这些开源模型能追平甚至局部超越闭源巨头？** 因为预训练配方公开、微调成本被 LoRA 这类技术打下来了，加上算法上的各种巧思——整个领域的护城河没有外界想象的那么深。
- **当你看到“我们用很少的成本微调出了一个专业领域模型”**——你现在知道，那大概率就是在某个开源底座上做了 LoRA 微调，而不是从零预训练。能分清这两者，你就比 90% 的吃瓜群众更懂行了。

自检

1. LoRA 凭什么能让微调"又快又省"? 它和完整微调的区别在哪?
2. 列出这本书故意没讲、但真实产品里很重要的至少 4 块拼图。
3. 这些拼图是"全新原理"还是"在主干上加的枝"?

第 8 天 综合：用底层原理，看懂整个 AI 世界

一句话直觉：你现在拥有了一套"第一性原理"。新闻里再花哨的 AI 名词，都能归进**四个抽屉**。会归类，你就不再被新词牵着走——这，就是这门课最终想给你的"根本性认识"。

对应书中

全书的综合 + 第 7.9.2 节（如何在快速发展的领域里保持判断）

核心：一张能装下整个 AI 世界的地图

任何一个 AI 新进展，几乎都属于下面四类之一。看到任何新词，先问"它属于哪个抽屉"：

抽屉 ① 更大/更好的预训练器（动的是"模型本身和它的训练"） **抽屉 ② 预训练之后，如何塑造它的行为**（动的是"微调/对齐"） **抽屉 ③ 推理时，如何给它更多上下文、工具、思考时间**（不改模型，改"怎么用它"） **抽屉 ④ 效率技巧**（让模型更小、更快、更便宜）

下面用你这 7 天学的原理，把当下最热的词逐个归位、各用两三句话拆穿：

热词	属于哪个抽屉	用你学过的原理，它其实是……
规模法则 / 更大的模型	①	同一块 Transformer 积木堆更多、喂更多数据、烧更多算力，损失可预测地下降。（第 4-5 天）
MoE（专家混合）	①④	把积木里的"前馈网络"换成很多并排的小专家，每个词只走几个——参数极大但计算不成比例增加。（第 4 天埋的钩子）
RLHF / 对齐	②	指令微调之后，再用"人类更喜欢哪个回答"的偏好信号继续调教行为。机制还是梯度更新，只是信号换成了"偏好"。（第 6 天）
推理模型（如 o 系列、DeepSeek-R1）	②③	训练/引导模型在回答前先生成一长串"思考草稿"，把更多计算花在推理时。本质还是预测下一个词，只是先预测一大段思考。（第 5 天解码 + 第 1 天主线）
RAG（检索增强）	③	不把知识全塞进参数，而是临时检索相关文本、塞进上下文窗口，让注意力去读它。（第 2 天上下文 + 第 3 天注意力）
智能体（Agent）	③	把模型放进一个循环：让它输出"动作"→执行→把结果喂回上下文→再决定下一步。仍然是"预测下一个 token"，只是这次 token 是动作。（第 1 天主线）

热词	属于哪个抽屉	用你学过的原理，它其实是……
多模态（看图、听声）	①③	把图像/音频也切成"token"、变成向量，和文字一起喂进同一个模型。一切皆向量。（第 2 天嵌入）
长上下文（128K / 1M）	④	想办法绕开注意力 n^2 的成本，让模型一次能读更多 token。（第 3 天的代价问题）
量化 / 蒸馏	④	量化=用更少的位数存参数（参数本就是数字）；蒸馏=训练一个小模型去模仿大模型。都是为了更小更快。（第 4 天"参数即数字"）
开源 vs 闭源（DeepSeek/Qwen/Llama vs GPT/Claude/Gemini）	全部	用的是同一套配方。开源能追平，正因为底层原理没有秘密——而这套原理你已经全程拆过了。

记住这张地图的终极用法：以后刷到任何 AI 新闻，先别管它包装得多唬人，问自己三个问题——

1. 它属于哪个抽屉？（是把模型做大、还是改调教、还是改用法、还是求效率？）
2. 它动了我学过的哪个零件？（嵌入？注意力？前馈？训练循环？微调？解码？）
3. 它是真·新原理，还是在已知主干上加的一根枝？（99% 是后者。）

能稳定回答这三个问题，你对 AI 的认识，就已经是"根本性"的了——不被热词裹挟，能看见骨架。

结课作业（Capstone，建议认真做）

不写代码，做一件更有价值的事：找三条最近一周的 AI 新闻/产品发布，每条用上面那张地图分析一遍——它属于哪个抽屉、动了哪个零件、是新原理还是加的枝。写下来。能把这三条讲清楚，你就真正完成了这门课的目标：不只是知道 LLM 怎么造，而是获得了看懂整个 AI 世界的底层视角。

怎样在这个飞速变化的领域里保持判断（送你的话）

2026 年这个领域几乎每周都有新模型发布，信息多到让人焦虑。但你现在有了"免疫力"：

- 底层原理变化极慢——Transformer、注意力、预训练+微调，这套骨架好几年没变。你学的是骨架，不是会过期的新闻。
- 新东西几乎都是已知零件的重新组合。你已经有了零件清单。
- 判断一个新进展重不重要，就看它在哪个抽屉、解决了什么真问题，而不是看它的发布会多热闹。

把"预测下一个词"这条主线、和这四个抽屉的地图带走。它们会比任何一个具体模型活得更久。

附：8 天速览

天	主题	一句话直觉	对应章节
1	大地图	LLM = 超级下一个词预测器	第 1 章 + 附录 A
2	文字变数字	意义即位置	第 2 章
3	注意力（心脏）	每个词环顾全场、拉取相关信息	第 3 章
4	拼装 GPT	一块积木重复堆 N 层	第 4 章
5	预训练	把半个互联网压进参数，自带答案	第 5 章
6	微调	从"会接话"到"听话"=ChatGPT 诞生	第 6–7 章
7	工程现实+缺失拼图	主干已成，其余皆是枝	附录 D/E + 7.9
8	综合	四个抽屉装下整个 AI 世界	全书综合

想压成 7 天：把第 7 天的 LoRA 并到第 6 天、缺失拼图的预告并到第 8 天即可。

贯穿全程的一句话，请带走：所有 AI，归根结底都在预测下一个词。